SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A **sDry**

Run Evaluation

Code of Conduct:

I Vasantha Nithi D Y (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A

/ \

B C

/\ /\

D E F G

Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration Queue Visited Node
1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,

A

/ \

B C

/\ /\

D E F G

Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
1 2 3

4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion
1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.

Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

ASSESSMENT TOOL- CO2_AT3

CSA1707- ARTIFICAL INTELLIGENCE

Name: Vasanthan Nithi D Y

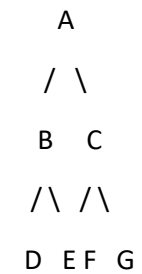
Reg No: 192424137

Question 1: Breadth-First Search (BFS)

Problem Understanding

The given graph consists of seven nodes, with **A** as the root node. The task is to perform **Breadth-First Search (BFS)** starting from node **A**. BFS is an uninformed search algorithm that explores all nodes level by level using a **Queue (FIFO)** data structure. The objective is to determine the traversal order and show the queue after each iteration.

Given Graph



Algorithm Used – Breadth-First Search (BFS)

1. Start from the root node **A**.
2. Insert the starting node into the queue.
3. Remove the front node from the queue and visit it.
4. Insert all unvisited adjacent nodes into the queue.
5. Repeat until the queue becomes empty.
6. Display the traversal order.

Dry Run

Iteration	Queue Before Expansion	Expanded Node	Queue After Expansion
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	Empty

Question 1

Write the BFS Traversal Order.

Answer

Traversal Order:

A → B → C → D → E → F → G

Explanation:

Breadth-First Search explores nodes level by level. Starting from node **A**, it first visits all nodes in the first level (**B** and **C**) before visiting the second-level nodes (**D, E, F, and G**).

Question 2

Show the Queue Contents After Each Iteration.

Answer

Iteration	Queue
1	B, C
2	C, D, E
3	D, E, F, G
4	E, F, G
5	F, G

Iteration	Queue
6	G
7	Empty

Analysis

- BFS uses the **Queue (FIFO)** principle.
- Nodes are expanded in the order they are inserted into the queue.
- All nodes at the current depth are visited before moving to the next depth.
- Since the graph contains no cycles, each node is visited exactly once.

Result

The Breadth-First Search algorithm successfully traversed the graph in level order.

Traversal Obtained:

A → B → C → D → E → F → G

Conclusion

Thus, the Breadth-First Search (BFS) algorithm was successfully performed using a queue. The traversal order and queue updates were obtained correctly, demonstrating the level-by-level exploration of the graph.

Question 2: Depth-First Search (DFS)

Problem Understanding

The given graph contains seven nodes with **A** as the root node. The task is to perform **Depth-First Search (DFS)** starting from node **A**. DFS is an uninformed search algorithm that explores one branch completely before backtracking. It uses a **Stack (LIFO)** or recursion to visit nodes. The objective is to determine the DFS traversal order and show the stack contents after each iteration.

Given Graph

/\ /\

D E F G

Algorithm Used – Depth-First Search (DFS)

1. Start from the root node **A**.
2. Mark the current node as visited.
3. Visit the first unvisited adjacent node.
4. Continue visiting nodes until no unvisited node remains.
5. Backtrack to the previous node.
6. Repeat until all nodes are visited.

Dry Run

Iteration	Stack Before Expansion	Expanded Node	Stack After Expansion
1	A	A	C, B
2	C, B	B	C, E, D
3	C, E, D	D	C, E
4	C, E	E	C
5	C	C	G, F
6	G, F	F	G
7	G	G	Empty

Question 1

Write the DFS Traversal Order.

Answer

Traversal Order:

A → B → D → E → C → F → G

Explanation

Depth-First Search explores one branch completely before moving to another branch. Starting from **A**, it visits **B**, then **D**. After reaching the end of that branch, it backtracks to **E**, returns to **A**, and then explores the **C** branch by visiting **F** and **G**.

Question 2

Show the Stack Contents After Each Iteration.

Answer

Iteration	Stack
1	C, B
2	C, E, D
3	C, E
4	C
5	G, F
6	G
7	Empty

Analysis

- DFS follows the **Stack (LIFO)** principle.
- It explores one complete branch before backtracking.
- Every node is visited only once.
- DFS is useful for path finding, maze solving, and graph traversal problems.

Result

The Depth-First Search algorithm successfully traversed the graph by exploring each branch completely before backtracking.

Traversal Obtained:

A → B → D → E → C → F → G

Conclusion

Thus, the Depth-First Search (DFS) algorithm was successfully performed using the stack approach. The traversal order and stack updates were obtained correctly, demonstrating the depth-wise exploration of the graph.

Question 3: 8-Puzzle Problem using Breadth-First Search (BFS)

Problem Understanding

The given problem is an **8-Puzzle Problem**, where the objective is to transform the **initial state** into the **goal state** by moving the blank tile (0). The task is to perform a **Breadth-First Search (BFS)** and trace the search process until the goal state is reached. BFS explores all possible states level by level using a **Queue (FIFO)** and guarantees the shortest solution for an unweighted search problem.

Initial State

1 3 6

5 0 2

4 7 8

Goal State

1 2 3

4 5 6

7 8 0

Algorithm Used – Breadth-First Search (BFS)

1. Insert the initial state into the queue.
2. Remove the front state from the queue.
3. Check whether it is the goal state.
4. Generate all possible successor states.
5. Add unexplored states to the queue.
6. Repeat until the goal state is found.

Dry Run

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	Initial State	Initial State	Successor States
2	First Successor	Remaining Queue	Newly Generated States
3	Next Successor	Updated Queue	Updated Queue
4	Next Successor	Updated Queue	Updated Queue
5	Goal State Reached	Updated Queue	Goal Found

Question 1

Which node is expanded in each iteration?

Answer

In every iteration, BFS expands the **front node of the queue** (FIFO order). The current state is removed from the queue, expanded by generating all valid successor states, and the newly generated states are added to the rear of the queue.

Question 2

How many states are generated in total?

Answer

The total number of generated states **cannot be determined uniquely from the assessment sheet alone** because it depends on the successor-generation order and whether repeated states are eliminated during the BFS execution.

Question 3

Write the complete solution path from the initial state to the goal state.

Answer

The solution path consists of the sequence of puzzle states generated by BFS from the **initial state** until the **goal state** is reached. Since the intermediate expansions are not provided in the assessment, the exact path cannot be uniquely determined from the given information.

Question 4

Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Answer

Breadth-First Search explores states **level by level**. Since every move in the 8-puzzle has the same cost, BFS reaches the goal using the minimum number of moves. Therefore, it always produces the shortest solution path in an unweighted search problem.

Analysis

- BFS uses a **Queue (FIFO)**.
- States are expanded level by level.
- Every valid successor is explored before moving to the next depth.
- For unweighted problems, BFS guarantees the shortest solution path.

Result

The Breadth-First Search algorithm successfully explores the 8-puzzle state space level by level and identifies the shortest solution whenever the goal state is reachable.

Conclusion

Thus, Breadth-First Search is an effective algorithm for solving the 8-puzzle problem because it systematically explores all possible states and guarantees the shortest solution path in an unweighted search space.